

Programming 2 — Complete Course Summary

PBA Applied Computer Science · 2025–2026

Table of Contents

1. [Object-Oriented Programming \(OOP\)](#)
 - 1.1 Properties
 - 1.2 Operator Overloading & Static Methods & Inheritance
 - 1.3 Abstract Methods & Abstract Properties
 - 1.4 Overriding & Super
 2. [Functional Programming \(FP\)](#)
 - 2.1 Higher-Order Functions & Nested Functions
 - 2.2 Lambdas & Comprehensions
 - 2.3 Iterables & Generators
 - 2.4 Recursion
 3. [Testing](#)
 - 3.1 Pytest Basics
 - 3.2 Fixtures & Advanced Assertions
 4. [Regular Expressions \(Regex\)](#)
 - 4.1 Basics
 - 4.2 Advanced Patterns
 5. [Data Processing](#)
 - 5.1 Pandas — Intro & Series
 - 5.2 Pandas — DataFrames
 - 5.3 Visualization with Matplotlib
-

1. Object-Oriented Programming

OOP is the paradigm of building programs around **objects** — things that hold data (attributes) and behaviour (methods). Python supports four core pillars:

Pillar	What it means
Encapsulation	Hide internal details; expose only what's needed
Inheritance	A child class reuses/extends a parent class
Polymorphism	Different classes can respond to the same method call differently

Pillar	What it means
Abstraction	Define <i>what</i> must exist without saying <i>how</i>

1.1 Properties

The problem: Private attributes need getter/setter methods for validation, but that creates messy syntax.

```
# Old way – works but feels clunky
tea.set_temperature(82)
tea.get_temperature()
```

The solution: Python's `@property` decorator lets you *access* a private attribute as if it were public, while still running validation behind the scenes.

```
class Tea:
    def __init__(self, name, temperature):
        self.name = name
        self.__temperature = temperature    # private

    @property
    def temperature(self):                  # getter – read like an attribute
        return self.__temperature

    @temperature.setter
    def temperature(self, value):          # setter – validates before writing
        if value <= 0 or value >= 100:
            raise ValueError("Tea must be liquid")
        self.__temperature = value

# Usage – looks like a plain attribute, but validation runs automatically
green = Tea("Green", 80)
print(green.temperature)    # 80
green.temperature = 82      # calls the setter
green.temperature = 200     # raises ValueError!
```

Think of it like: a vending machine slot. You put money in and get a drink out — you don't see (or need to touch) the internal mechanism.

1.2 Operator Overloading, Static Methods & Inheritance

Operator Overloading

Python lets you define what `+`, `-`, `*`, `==`, `str()`, etc. mean **for your own classes** using **dunder (double-underscore) methods**.

Operator	Dunder method
<code>+</code>	<code>__add__</code>
<code>-</code>	<code>__sub__</code>
<code>*</code>	<code>__mul__</code>

Operator	Dunder method
/	<code>__truediv__</code>
==	<code>__eq__</code>
<code>str()/print()</code>	<code>__str__</code>

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)    # Vector(4, 6) – uses __add__ then __str__
```

Think of it like: teaching Python what "adding two shopping carts" means for your specific domain.

Static Methods

A **static method** belongs to the *class*, not to any specific object. It has no `self` parameter.

```
class FootballTeam:
    def __init__(self, name, points):
        self.name = name
        self.points = points

    @staticmethod
    def compare_teams(team1, team2):
        return team1.points > team2.points    # no self needed

team_a = FootballTeam("Ajax", 15)
team_b = FootballTeam("Anderlecht", 12)
FootballTeam.compare_teams(team_a, team_b)    # True – called on the CLASS
```

When to use: utility/helper functions that logically belong to a class but don't need to read or write instance data.

Inheritance

A **child class** inherits all attributes and methods from a **parent class** and can add or override them.

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):    # Dog inherits from Animal
    def bark(self):
```

```

        print("Woof!")

my_dog = Dog()
my_dog.speak()    # inherited from Animal
my_dog.bark()     # defined in Dog

```

Think of it like: a Car is a more specific kind of Vehicle. It shares everything a vehicle has (wheels, engine) and adds its own features (GPS, air conditioning).

1.3 Abstract Methods & Abstract Properties

An **abstract class** is a *blueprint* — it cannot be instantiated directly. It forces every subclass to implement certain methods.

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def calculate_area(self):
        pass # no implementation here!

# Shape() ← This would raise a TypeError

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def calculate_area(self):
        return 3.14 * self.radius ** 2 # must implement it

c = Circle(7)
print(c.calculate_area()) # 153.86

```

Abstract Properties work the same way but for attributes:

```

class Shape(ABC):
    @property
    @abstractmethod
    def color(self):
        pass # order matters! @property must be on top

```

Think of it like: a job description. It says "this role **MUST** include Python skills and team meetings" — but the actual person doing the job fills in the details.

Key rule: If a subclass doesn't implement *all* abstract methods/properties, Python raises a `TypeError` when you try to instantiate it.

1.4 Overriding & super ()

Overriding: A subclass provides its *own* version of a method that already exists in the parent.

```

class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):

```

```
def speak(self):          # overrides Animal's speak()
    print("Dog barks")
```

```
Dog().speak()    # "Dog barks" – the subclass version wins
```

super() : Lets a subclass call the *parent's* version of the method (in addition to, or instead of, its own).

```
class Dog(Animal):
    def speak(self):
        super().speak()    # runs Animal.speak() first
        print("Dog barks") # then adds its own line
```

```
Dog().speak()
# Animal speaks
# Dog barks
```

Think of it like: a new employee who follows the company standard procedure (`super()`) but also adds their own personal touch.

Method Resolution Order (MRO): Python uses MRO to determine *which* parent method to call when there's multiple inheritance. You can inspect it with `ClassName.__mro__`.

2. Functional Programming

Functional Programming (FP) treats **functions as first-class citizens** — they can be stored in variables, passed as arguments, and returned from other functions.

2.1 Higher-Order Functions & Nested Functions

A **higher-order function** is a function that:

- accepts another function as an argument, OR
- returns a function as its result.

```
# Built-in example: sorted() accepts a function as 'key'
movies = ["The Matrix", "Interstellar", "Jaws"]
sorted_by_length = sorted(movies, key=len)
# ['Jaws', 'The Matrix', 'Interstellar']
```

```
# Custom higher-order function
def count_matching(items, condition):
    return sum(1 for item in items if condition(item))

def is_long(s):
    return len(s) > 5

count_matching(movies, is_long)    # 2
```

Nested functions: A function defined inside another function. It has access to the outer function's variables.

```
def make_multiplier(factor):
    def multiply(x):          # nested – uses 'factor' from outer scope
```

```
        return x * factor
    return multiply

triple = make_multiplier(3)
triple(10)    # 30
```

Think of it like: a factory that produces specialised tools. `make_multiplier(3)` produces a "triple-it" tool.

2.2 Lambdas & Comprehensions

Lambda Functions

A **lambda** is a compact, anonymous (no-name), one-line function.

```
# Regular function
def add(x, y):
    return x + y

# Same thing as a lambda
add = lambda x, y: x + y

# Most useful when passed directly to a higher-order function
numbers = [3, 1, 4, 1, 5, 9]
sorted(numbers, key=lambda x: -x)    # sort descending: [9, 5, 4, 3, 1, 1]
```

Limits: lambdas can only contain a single *expression* — no `if/else` blocks, no loops, no assignments.

List Comprehensions

A concise way to build a list by combining a loop and an optional filter in one line.

```
# Old way
squares = []
for n in range(1, 6):
    squares.append(n ** 2)

# List comprehension
squares = [n ** 2 for n in range(1, 6)]
# [1, 4, 9, 16, 25]

# With filter
even_squares = [n ** 2 for n in range(1, 6) if n % 2 == 0]
# [4, 16]
```

Dictionary comprehension:

```
word_lengths = {word: len(word) for word in ["hello", "world", "python"]}
# {'hello': 5, 'world': 5, 'python': 6}
```

Think of it like: a food processor — instead of chopping each vegetable one at a time, you describe the rule once and get the result in one go.

2.3 Iterables & Generators

Iterable: Any object you can loop over — lists, tuples, dicts, sets, strings, etc.

Generator Functions

A **generator** is like a function that can *pause* and *resume*. It uses `yield` instead of `return`.

```
def count_up(limit):
    n = 0
    while n < limit:
        yield n          # pauses here and gives back n
        n += 1           # resumes from here next time

for number in count_up(5):
    print(number)        # 0 1 2 3 4
```

Why generators? They are *lazy* — they produce values one at a time instead of storing everything in memory.

```
# List: stores ALL 1 million numbers in RAM immediately
big_list = [x for x in range(1_000_000)]

# Generator: computes values on demand – tiny memory footprint
big_gen = (x for x in range(1_000_000))  # generator comprehension
```

next () function: Manually pull one value from a generator.

```
gen = count_up(3)
next(gen)    # 0
next(gen)    # 1
next(gen)    # 2
next(gen)    # StopIteration error – generator is exhausted
```

Think of it like: a conveyor belt in a factory. Items are produced and sent forward one at a time — you don't need a warehouse to store them all first.

2.4 Recursion

Recursion: A function that calls itself. Every recursive function needs:

1. A **base case** — the simple condition that stops the recursion.
2. A **recursive case** — the function calls itself with a modified argument, moving toward the base case.

```
def factorial(n):
    # Base case: factorial of 1 is 1
    if n == 1:
        return 1
    # Recursive case: n! = n × (n-1)!
    return n * factorial(n - 1)

factorial(5)
# 5 × 4 × 3 × 2 × 1 = 120
```

Visualising the call stack:

```
factorial(5)
→ 5 * factorial(4)
   → 4 * factorial(3)
      → 3 * factorial(2)
         → 2 * factorial(1)
            → 1 ← base case!
```

When to use recursion:

- Navigating tree/nested structures (e.g. folder hierarchies, family trees)
- Depth-First Search (DFS) on graphs
- Problems with a naturally self-similar structure (fractals, parsing)

When to prefer a loop: simple counting, summing — loops are faster and use less memory (each recursive call adds a frame to the call stack).

Think of it like: Russian nesting dolls. To open the outermost doll, you open the next one inside... until you reach the tiny solid one that can't be opened (the base case).

3. Testing

Testing is how you verify that your code does what it's supposed to do — automatically, repeatedly, and without manual checking.

What makes a good test?

- Automated (no human needed)
 - Fine-grained (tests one thing at a time)
 - Readable (clear what's being tested)
 - Fast
 - Isolated (doesn't depend on other tests)
-

3.1 Pytest Basics

Pytest is Python's go-to testing framework. Any function starting with `test_` is automatically collected and run.

```
# In tests.py
def add(x, y):
    return x + y

def test_add_two_positives():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-1, 1) == 0
```

Run with: `pytest`

Parameterized Tests

Test the same function with many different inputs without copy-pasting:

```
import pytest

@pytest.mark.parametrize('a, b, expected', [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
    (100, -50, 50),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Floating-Point Comparison with approx

`0.1 + 0.2` is not exactly `0.3` in floating-point arithmetic. Use `approx`:

```
from pytest import approx

def test_floating_point():
    assert 0.1 + 0.2 == approx(0.3)          # passes!
    assert 0.1 + 0.2 == approx(0.3, abs=0.01) # with tolerance
```

Testing Exceptions

```
import pytest

def test_division_by_zero():
    with pytest.raises(ZeroDivisionError):
        result = 1 / 0    # this line should raise the error
```

3.2 Fixtures & Advanced Assertions

Setup and Teardown

When tests need shared preparation (e.g. creating a database), use `setup_function` and `teardown_function`:

```
def setup_function():
    print("runs BEFORE each test")

def teardown_function():
    print("runs AFTER each test")

def test_one():
    assert 1 + 1 == 2
```

Fixtures (Better Alternative)

Fixtures are reusable setup functions. A test *declares* what it needs as a parameter, and pytest provides it automatically.

```
import pytest

@pytest.fixture
def sample_user():
```

```

    return {"name": "Alice", "age": 25}

def test_user_name(sample_user):    # pytest sees the parameter name and calls
the fixture
    assert sample_user["name"] == "Alice"

def test_user_age(sample_user):
    assert sample_user["age"] == 25

```

Fixtures with teardown — use `yield` to separate setup from cleanup:

```

@pytest.fixture
def db_connection():
    conn = open_database()    # setup
    yield conn                # test runs here
    conn.close()              # teardown – runs after the test

```

Think of it like: a restaurant that sets the table (fixture setup) before each customer, and clears it (teardown) after they leave.

4. Regular Expressions

A **regular expression (regex)** is a pattern-matching language. It lets you search, validate, and transform strings based on rules rather than exact text.

Python's `re` module provides all the tools.

4.1 Basics

Core Functions

Function	What it does
<code>re.match(pattern, string)</code>	Match at the start of the string only
<code>re.fullmatch(pattern, string)</code>	Match the entire string
<code>re.search(pattern, string)</code>	Find the pattern anywhere in the string
<code>re.findall(pattern, string)</code>	Return a list of all matches
<code>re.sub(pattern, replacement, string)</code>	Replace matches with new text

Core Pattern Symbols

Pattern	Meaning	Example
<code>.</code>	Any character (except newline)	<code>a.c</code> matches <code>abc</code> , <code>a1c</code> , <code>a-c</code>
<code>^</code>	Start of string	<code>^Hello</code> matches strings starting with "Hello"
<code>\$</code>	End of string	<code>world\$</code> matches strings

Pattern	Meaning	Example
		ending with "world"
*	0 or more of the previous	ab* matches a, ab, abb, abbb
+	1 or more of the previous	ab+ matches ab, abb but NOT a
?	0 or 1 of the previous	colou?r matches color and colour
{m}	Exactly m repetitions	\d{4} matches exactly 4 digits
[abc]	Any one character from the set	[aeiou] matches any vowel
(...)	Group	(ab)+ matches ab, abab
\	Or	cat\ dog matches "cat" or "dog"

```
import re

# Does this string start with a digit?
re.match(r'\d', "3 apples")    # Match!
re.match(r'\d', "apples")      # None

# Find all words in a sentence
re.findall(r'\w+', "Hello, World! 123")
# ['Hello', 'World', '123']

# Replace all digits with #
re.sub(r'\d', '#', "My PIN is 1234")
# 'My PIN is ####'
```

Tip: Always use raw strings `r'...'` for regex patterns to avoid backslash confusion.

4.2 Advanced Patterns

Predefined Character Classes

Shorthand	Equivalent	Meaning
\d	[0-9]	Any digit
\D	[^0-9]	Any non-digit
\w	[a-zA-Z0-9_]	Any word character
\W	[^a-zA-Z0-9_]	Any non-word character
\s	[\t\n\r]	Any whitespace
\S	[^ \t\n\r]	Any non-whitespace

Subgroups (Capturing Groups)

Parentheses () capture part of a match so you can extract it:

```
# Extract area code and number from a phone number
match = re.search(r'\((\d{3})\) \s*(\d{3}-\d{4})', "(555) 123-4567")
match.group(0)    # '(555) 123-4567'    - whole match
match.group(1)    # '555'              - first group
match.group(2)    # '123-4567'         - second group
```

Backreferences

Refer back to a previously captured group *within the same pattern*:

```
# Find doubled words like "the the" or "is is"
re.search(r'\b(\w+)\s+\1\b', "this is is a test")
# Matches "is is" - \1 refers back to the first captured group
```

Practical Example: Validate an Email

```
pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
re.fullmatch(pattern, "user@example.com")    # Match!
re.fullmatch(pattern, "not-an-email")        # None
```

5. Data Processing

Python's **Pandas** library is the standard tool for loading, cleaning, analysing, and transforming structured data.

5.1 Pandas — Intro & Series

Install once: `pip install pandas matplotlib`

Import convention:

```
import pandas as pd
import matplotlib.pyplot as plt
```

Pandas Series

A **Series** is a one-dimensional labeled array — like a single column from a spreadsheet.

```
temperatures = pd.Series([22, 18, 25, 30, 19],
                          index=["Mon", "Tue", "Wed", "Thu", "Fri"])

temperatures["Wed"]    # 25
temperatures.mean()    # 22.8
temperatures.max()     # 30
temperatures[temperatures > 20]    # filter: Mon (22), Wed (25), Thu (30)
```

Create from a dictionary:

```
scores = pd.Series({"Alice": 85, "Bob": 72, "Charlie": 91})
```

Useful Series methods: `.mean()`, `.sum()`, `.min()`, `.max()`, `.count()`,

```
.sort_values(), .value_counts()
```

5.2 Pandas — DataFrames

A **DataFrame** is a two-dimensional table — rows and columns, like a spreadsheet or database table.

Creating a DataFrame

```
# From a dictionary — keys = column names, values = column data
data = {
    "Name":    ["Alice", "Bob", "Charlie"],
    "Age":     [25, 30, 22],
    "Score":   [85, 72, 91]
}
df = pd.DataFrame(data)
```

Output:

	Name	Age	Score
0	Alice	25	85
1	Bob	30	72
2	Charlie	22	91

Accessing Data

```
df["Name"]          # a whole column (returns a Series)
df[["Name", "Score"]] # multiple columns (returns a DataFrame)
df.loc[0]           # row by label/index
df.iloc[1]          # row by integer position
df.loc[0, "Score"]  # single cell: row 0, column "Score"
```

Filtering Rows

```
# Students with score > 80
df[df["Score"] > 80]

# Multiple conditions
df[(df["Score"] > 80) & (df["Age"] < 30)]
```

Common Operations

```
df.shape          # (3, 3) — rows × columns
df.head()         # first 5 rows
df.tail()         # last 5 rows
df.describe()     # summary statistics
df.sort_values("Score", ascending=False) # sort by score

# Add a new column
df["Passed"] = df["Score"] >= 75

# Rename columns
df.rename(columns={"Score": "Grade"}, inplace=True)
```

Reading from Files

```
df = pd.read_csv("students.csv")
df = pd.read_excel("data.xlsx")
```

Think of it like: an Excel spreadsheet, but fully programmable and capable of handling millions of rows without breaking a sweat.

5.3 Visualization with Matplotlib

Matplotlib turns your data into charts and graphs.

The Two Core Concepts

- **Figure** — the whole canvas/window (can contain multiple charts)
- **Axes** — a single chart area with its own x- and y-axes

```
fig, ax = plt.subplots() # create a Figure with one Axes
ax.plot([1, 2, 3, 4], [1, 4, 2, 3])
plt.show()
```

Common Plot Types

```
# Line chart
ax.plot(x, y, color='blue', linewidth=2, label='Temperature')

# Bar chart
ax.bar(categories, values, color='steelblue')

# Scatter plot
ax.scatter(x, y, color='red', marker='o')

# Histogram
ax.hist(data, bins=20, color='green', edgecolor='black')
```

Labels & Formatting

```
ax.set_title("Monthly Temperatures")
ax.set_xlabel("Month")
ax.set_ylabel("°C")
ax.legend() # show the legend
plt.tight_layout() # auto-fix spacing
plt.savefig("chart.png") # save to file
plt.show() # display
```

Multiple Subplots

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4)) # 1 row, 2 columns
axes[0].plot(x1, y1)
axes[1].bar(categories, values)
```

Plotting directly from Pandas

Pandas DataFrames have built-in `.plot()` methods that use Matplotlib under the hood:

```
df["Score"].plot(kind="hist", bins=10)
df.plot(x="Name", y="Score", kind="bar")
plt.show()
```

Think of it like: Pandas is your data analyst who crunches the numbers, and Matplotlib is the graphic designer who turns those numbers into beautiful visuals.

Quick Reference Cheat Sheet

OOP

```
@property                # getter decorator
@property.setter          # setter decorator
@staticmethod             # no self – belongs to class
from abc import ABC, abstractmethod
class MyClass(ABC):
    @abstractmethod
    def my_method(self): pass    # must implement in subclass
super().method()             # call parent's version
def __add__(self, other): ...  # overload the + operator
def __str__(self): ...        # overload str() / print()
```

Functional Programming

```
lambda x, y: x + y          # anonymous function
[x**2 for x in range(10) if x % 2 == 0]  # list comprehension
{k: v for k, v in pairs}    # dict comprehension
(x for x in range(10))      # generator expression
sorted(items, key=lambda x: x.name)    # HOF with lambda
def gen(): yield 1; yield 2    # generator function
```

Testing

```
import pytest
def test_something(): assert result == expected
@pytest.mark.parametrize('a,b', [(1,2),(3,4)])
def test_fn(a, b): ...
with pytest.raises(ValueError): risky_call()
from pytest import approx
assert 0.1 + 0.2 == approx(0.3)
@pytest.fixture
def my_data(): return {...}
```

Regex

```
import re
re.match(r'\d+', text)      # match at start
re.search(r'\d+', text)     # search anywhere
re.findall(r'\d+', text)    # all matches as list
re.sub(r'\d', '#', text)    # replace matches
# Symbols: . ^ $ * + ? {m} [] () | \d \w \s
```

Pandas & Matplotlib

```
import pandas as pd, matplotlib.pyplot as plt
s = pd.Series([1,2,3])
df = pd.DataFrame({"A": [1,2], "B": [3,4]})
df["A"]                # column
df[df["A"] > 1]         # filter
df.sort_values("A")     # sort
fig, ax = plt.subplots()
ax.plot(x, y); ax.set_title("..."); plt.show()
```